Industrial Internship Report on

” Banking Information System”

Prepared by

Anshul

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time. **My Project: Banking Information System** My project, *Banking Information System*, was developed using Core Java to manage basic banking operations through a menu-driven, console-based application. It allows users to create bank accounts, deposit and withdraw money, transfer funds between accounts, check account balances, and view individual or all account details. The project uses object-oriented programming concepts, ArrayList for account management, and input validation to ensure accurate and reliable operations. Developing this project enhanced my Java programming skills, logical thinking, and practical understanding of software development and banking-related applications. Overall, it was a valuable learning experience that helped me apply theoretical knowledge to a practical programming project. .

TABLE OF CONTENTS

1 Preface 3

2 Introduction 5

2.1 About UniConverge Technologies Pvt Ltd 5

2.2 About upskill Campus 9

2.3 Objective 11

2.4 Reference 11

2.5 Glossary 12

3 Problem Statement 13

4 Existing and Proposed solution 14

5 Proposed Design/ Model 16

5.1 High Level Diagram (if applicable) 16

5.2 Low Level Diagram (if applicable) 17

5.3 Interfaces (if applicable) 17

6 Performance Test 18

6.1 Test Plan/ Test Cases 18

6.2 Test Procedure 19

6.3 Performance Outcome 19

7 My learnings 21

8 Future work scope 22

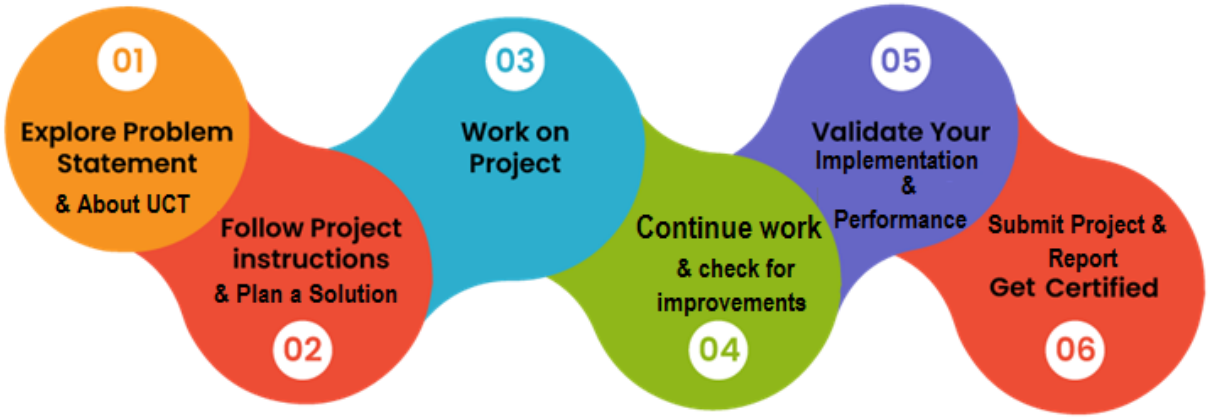
1 Preface

Summary of the whole 6 weeks' work.

I spent these six weeks working on a Core Java project, the Banking Information System. In the first week I read the problem statement and learned about UCT and upskill Campus. In the next two weeks I revised my Java basics and planned how the program would work. After that I wrote the code in small parts, tested it with right and wrong inputs, and fixed the mistakes I found. In the last week I prepared this report and uploaded my code and report to GitHub.

Need of a relevant internship in career development

Classroom study teaches the concepts, but an internship shows how to use them on a real task with a deadline. As a second-year student at AKTU, I needed this practice. It helped me understand what companies expect from a fresher, such as clean code, proper testing and clear documentation. A relevant internship also gives you something real to show in your resume and in interviews.



Brief about my project and problem statement

The task was to build a console-based Banking Information System in Core Java. The program lets a user create an account, deposit and withdraw money, transfer money between accounts, check the balance, and view account details. It also has to handle wrong inputs properly, such as an invalid phone number, a negative amount, or a withdrawal larger than the balance.

Opportunity given by USC/UCT

upskill Campus and UniConverge Technologies gave me a chance to work on an industry-style project instead of only reading theory. The project had a clear problem statement, a fixed report format and a

GitHub submission, which is how real teams share their work. This gave me my first experience of finishing a project from start to end.

How the program was planned

The program was planned in six steps. First, I explored the problem statement and learned about UCT. Second, I followed the project instructions and planned a solution. Third, I worked on the project. Fourth, I continued the work and checked for improvements. Fifth, I validated my implementation and its performance. Sixth, I submitted the project and report.

My learnings and overall experience

I understood object-oriented programming much better by actually using classes, objects and methods in a working program. I learned to check every input, to break a big problem into small methods, and to test my own code. I also learned to use GitHub. At first I made many small errors, and fixing them taught me patience. Overall it was a good and useful experience, and it made me more confident about programming.

Thanks

I sincerely thank [Mentor/Trainer name] for guiding me and answering my doubts during the internship. I also thank the upskill Campus and UniConverge Technologies team for this opportunity, [Teacher name] and my teachers at AKTU for the knowledge that helped me, and my family and friends for their constant support.

Message to my juniors and peers

Start early and do not leave the work for the last week. Write the code yourself and make sure you can explain every line of it. Test your program properly, because small mistakes show up only when you try different inputs. Treat the internship project seriously, because it is something you can proudly show in your resume.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



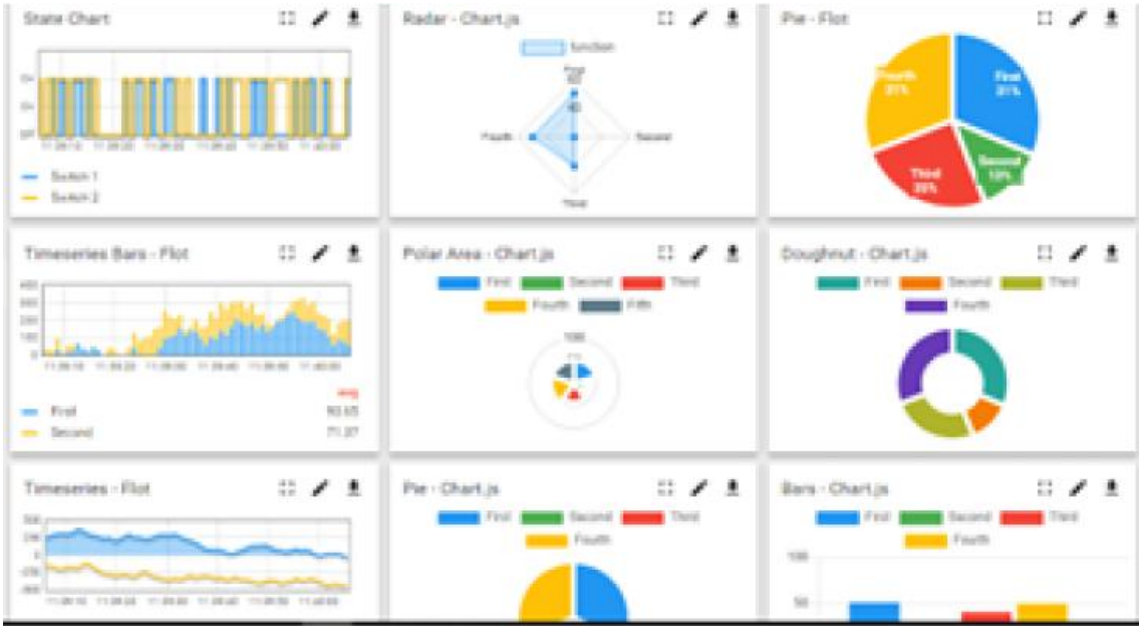
i. UCT IoT Platform (UCT Insight)

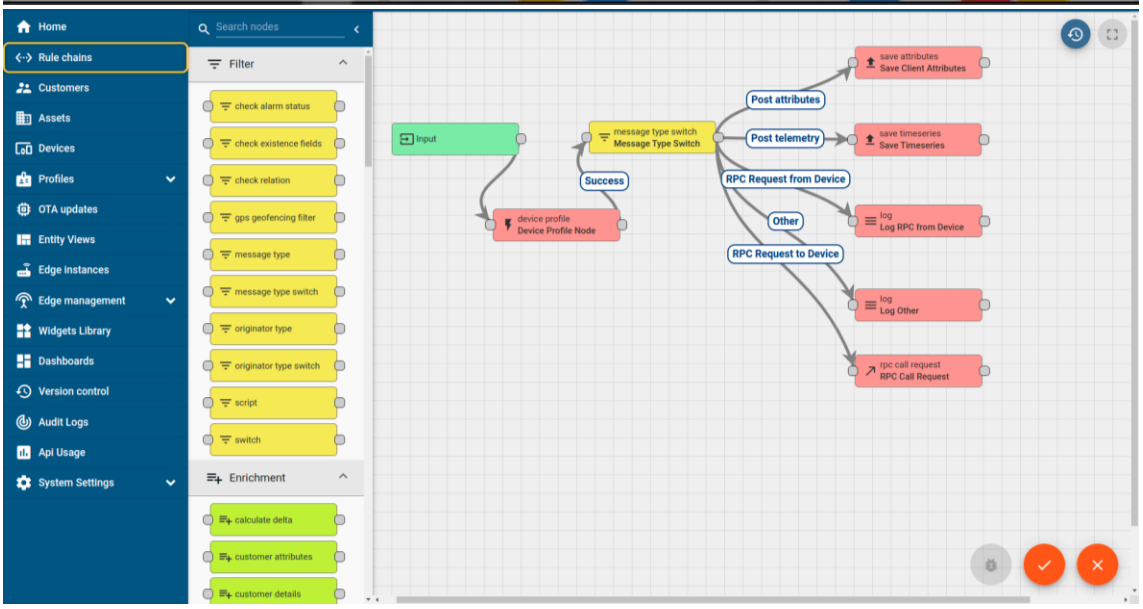
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

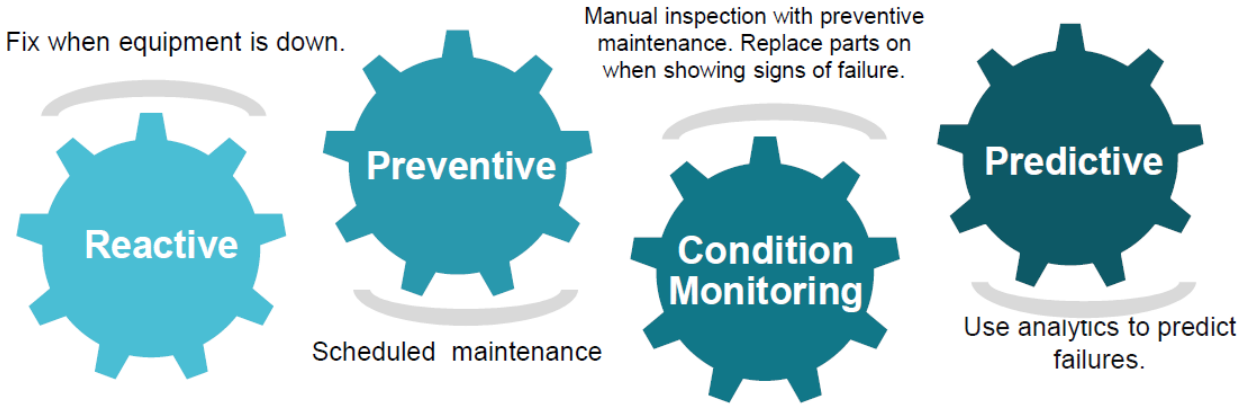


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

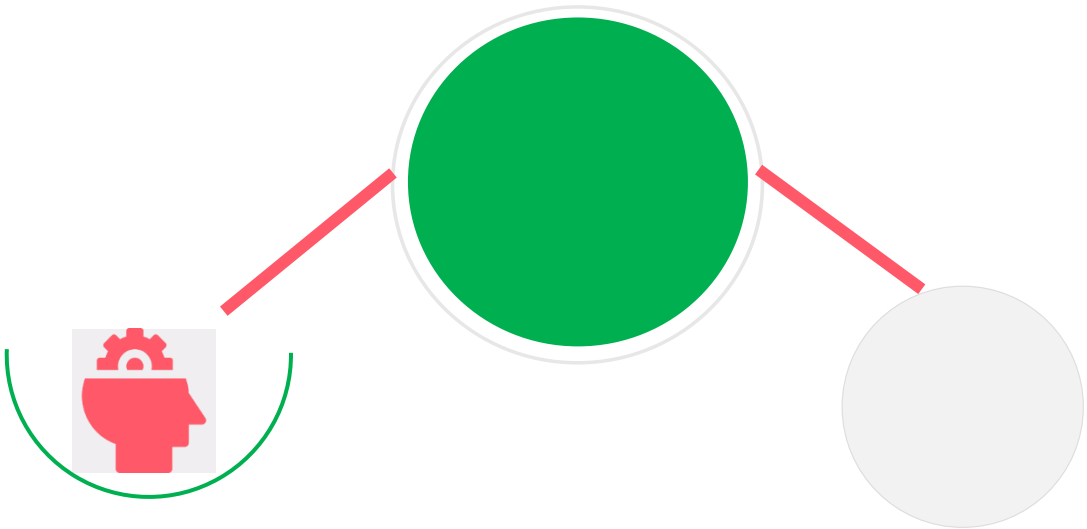
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

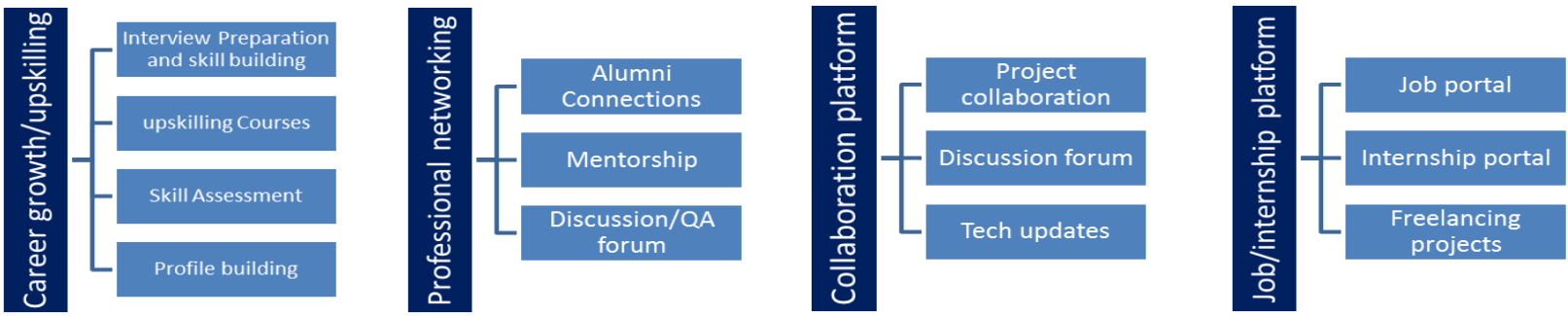
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

The following references are relevant to the Java concepts used in this project:

[1] Oracle, “The Java Tutorials — Learning the Java Language.” https://docs.oracle.com/javase/tutorial/

[2] Oracle Java SE API, “ArrayList Class.” https://docs.oracle.com/en/java/javase/

[3] Oracle Java SE API, “Scanner Class.” https://docs.oracle.com/en/java/javase/

2.6 Glossary

Term	Acronym / Meaning
BIS	Banking Information System
OOP	Object-Oriented Programming
JDK	Java Development Kit
JVM	Java Virtual Machine
CLI	Command-Line Interface
ArrayList	A resizable Java collection used to store Account objects
Input validation	Checking user-provided values before processing them
Account number	A unique number assigned to each account during the current program run

3 Problem Statement

In the assigned problem statement

Basic banking operations require an organized way to create account records and perform common transactions. For a small learning project, the problem is to build a simple application that can maintain account details and support deposits, withdrawals, transfers, balance checks, and account-information display through a clear menu. The application must also handle invalid input, unknown account numbers, non-positive transaction amounts, and attempts to withdraw or transfer more money than the sender has available.

The supplied project addresses this problem with a Java console interface and in-memory account storage. It is a learning prototype, not a production banking platform, and it does not connect to a real bank or persist records in a database.

4 Existing and Proposed solution

4.1 Existing Solutions and Their Limitations

Banking systems commonly use database-backed applications with authentication, audit logs, transaction processing, and secure interfaces. Such systems are designed for reliability and protection of customer data. A simple console-based learning application is smaller and easier to understand, but it has important limitations: account data is stored only in memory, there is no login or role-based access, and it does not provide encryption, audit trails, database transactions, or integration with real banking services.

4.2 Proposed Solution

The proposed solution is a menu-driven Banking Information System written in Core Java. It uses an Account class to store an account number, holder name, phone number, and balance. An ArrayList stores the accounts while the program is running. A menu lets the user create accounts, deposit money, withdraw money, transfer funds, check balances, view account details, view all accounts, or exit.

4.3 Value Addition

Automatically assigns account numbers beginning at 1001 during each program run.

Validates that a customer name is not empty and that the phone number contains exactly 10 digits.

Requires an opening balance of at least Rs. 500.

Rejects zero or negative transaction amounts and checks for insufficient balance before withdrawal or transfer.

Checks whether an account exists and prevents transfers from an account to itself.

Displays formatted balances and clear success or error messages

4.1 Code submission (Github link):

https://github.com/Bobby8122/upskillcampus/blob/main/BankingInformationSystem.java

4.2 Report submission (Github link) :

https://github.com/Bobby8122/upskillcampus/blob/main/BankingInformationSystem.java

5 Proposed Design/ Model

The program is divided into two main parts. The Account class represents an individual account and provides methods to access account information, deposit money, withdraw money, and display details. The BankingInformationSystem class contains the main menu, account collection, input-reading helpers, account search, and the methods that coordinate each banking operation.

5.1 High Level Diagram (if applicable)

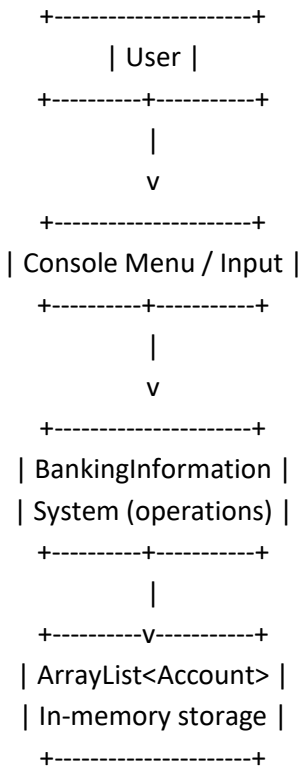


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

Start
|
Display menu and read choice
|
+-- 1: Create account --> Validate name, phone, opening balance --> Store account
+-- 2: Deposit -------> Find account --> Validate amount --> Add to balance
+-- 3: Withdraw ------> Find account --> Check balance --> Deduct if sufficient
+-- 4: Transfer ------> Find both accounts --> Validate amount --> Withdraw + deposit
+-- 5: Check balance -> Find account --> Display balance
+-- 6: Account details -> Find account --> Display account information
+-- 7: All accounts --> Display all stored accounts
+-- 8: Exit ----------> Close input scanner and end
|
Repeat menu unless choice is 8

5.3 Interfaces (if applicable)

The current interface is text-based and runs in a terminal or IDE console. It displays a numbered menu and prompts the user to enter the required values. Output messages confirm successful operations or explain errors. The project does not currently include a graphical user interface, web interface, external API, database connection, or network protocol.

Main interface options:

1. Create new account
2. Deposit money
3. Withdraw money
4. Transfer money
5. Check balance
6. View account details
7. View all accounts
8. Exit

5.4 Performance Test

The main constraints identified from the code are correctness of transaction calculations, validation of user input, account lookup time, and temporary in-memory storage. Account lookup uses a sequential search through the ArrayList, so lookup time grows with the number of stored accounts. The project has no persistent database, so all account records are lost when the program exits. No formal benchmark results were supplied; therefore, the test cases and outcomes below describe expected behavior and should be verified by running the program.

5.4 Test Plan/ Test Cases

Test ID	Test Input / Scenario	Expected Result	Status
TC-01	Create account with valid name, 10-digit phone, and Rs. 500 or more	Account created and account number displayed	To be tested
TC-02	Enter blank customer name	Program asks for a non-empty name	To be tested
TC-03	Enter phone number with fewer or more than 10 digits	Program rejects it and asks again	To be tested
TC-04	Enter opening balance below Rs. 500	Program asks for an eligible opening balance	To be tested
TC-05	Deposit a positive amount into an existing account	Balance increases by the deposit amount	To be tested
TC-06	Withdraw an amount within the available balance	Balance decreases by the withdrawal amount	To be tested
TC-07	Withdraw more than the available balance	Transaction is rejected with an insufficient-balance message	To be tested
TC-08	Transfer a valid amount between two different	Sender balance decreases and receiver	To be tested

	accounts	balance increases	
TC-09	Use an unknown account number	Program displays that no account was found	To be tested
TC-10	Enter zero or a negative transaction amount	Program rejects the amount and asks again	To be tested
TC-11	Select an invalid menu option	Program displays an invalid-choice message	To be tested
TC-12	Choose Exit	Program displays goodbye message and terminates the menu loop	To be tested

5.5 Test Procedure

Compile BankingInformationSystem.java using a Java Development Kit (JDK) compatible with the source code.

Run the program in a terminal or IDE console.

Select each menu option and provide valid and invalid inputs according to the test plan.

Record the displayed messages and account balances before and after each transaction.

Verify that rejected operations do not change balances and that valid deposits, withdrawals, and transfers update the relevant balances correctly.

Update the Status column in the test table with Pass or Fail after performing each test. Do not mark a test as passed until it has actually been run.

5.6 Performance Outcome

Based on code inspection, the program includes basic safeguards for invalid numeric input, invalid amounts, missing accounts, insufficient funds, and transfers to the same account. These checks are intended to reduce common input and transaction errors. Actual execution time, memory usage, and maximum account capacity have not been measured. For larger datasets, a database and indexed account lookup would be more appropriate. For real-world banking use, authentication, authorization,

persistent storage, secure handling of personal information, transaction logging, and stronger transaction guarantees would be required.

6 My learnings

While developing the Banking Information System, I learned how to organize a Java program into classes and methods, create objects, use encapsulation through private fields and getter methods, and store objects in an ArrayList. I practiced taking input with Scanner, handling invalid numeric input with exception handling, using loops and switch statements to build a menu-driven program, and implementing validation for banking operations. I also improved my understanding of balance calculations, account searching, error messages, and the importance of testing both valid and invalid scenarios. These skills will support my future growth in Java development, debugging, problem solving, and building more complete software applications

7 Future work scope

- Save accounts in a file or a database (for example MySQL) so data is not lost when the program closes.
- Add login with account number and PIN for security.
- Keep a transaction history and generate a mini statement for each account.
- Add interest calculation, different account types and an account closing option.
- Build a graphical interface (Swing or JavaFX) or a web version.
- Use BigDecimal for exact money calculations and add unit tests with JUnit.